AI Homework: Sokoban Solver

Christian Carrillo
Matricola: 2017626

1 Introduction

This report documents the design, implementation, and analysis of an automated pipeline for solving the **Sokoban** puzzle, a classic transport puzzle where a player pushes boxes to storage locations in a grid. The objective of this project was to model the problem as a search environment and solve it using two distinct Artificial Intelligence techniques:

1. **Search-Based Solver (Task 2.1):** A custom implementation of the A^* algorithm, enhanced with duplicate elimination and multiple heuristic functions.
2. **Automated Planning (Task 2.2):** A reduction of the Sokoban problem to PDDL (Planning Domain Definition Language), solved using General Purpose Planners (Fast Downward and LAMA).

1.1 Problem Choice

I chose **Sokoban** because it represents a challenging standard in AI research. It requires reasoning about constraints (walls, blocked paths), deadlocks, and long-term planning, making it an excellent candidate for comparing heuristic search against domain-independent planning. The problem is modeled as a set of grid-based instances of increasing difficulty, allowing for scalability analysis as required by the assignment.

1.2 Summary of Approach

The solution is structured as a modular Python pipeline. The core components are:

- **State Representation:** A canonical representation of the game state (player position and box coordinates) that detects static deadlocks to prune the search space early.
- **A* Implementation:** A robust A^* search that supports plug-and-play heuristic functions. I implemented and compared seven different heuristics, ranging from simple *Manhattan Distance* to complex assignments like *Hungarian Algorithm* and *Real Maze Distance*.
- **PDDL Generator:** An automated transpiler that converts game levels into `.pddl` problem files, allowing seamless integration with external planners.

2 Task 1: Problem Description and Modeling

2.1 Problem Description: Sokoban

For this assignment, I selected Sokoban, a classic transport puzzle where the player pushes boxes around a grid to place them onto designated storage locations. Formally, the state space can be modeled as a directed graph $G = (V, E)$, where V represents the set of game states (positions of the agent and boxes) and E represents the transitions defined by valid box pushes.

While the rules are deceptively simple—moves are deterministic and fully observable—Sokoban is computationally profound. It has been proven to be PSPACE-complete, implying that the difficulty of solving optimal instances grows exponentially with the problem size.

2.1.1 The Nature of Difficulty

The difficulty of Sokoban stems from several distinct factors that challenge both human intuition and standard artificial intelligence search algorithms:

Irreversibility and Deadlocks The most defining characteristic of Sokoban is the irreversibility of mistakes. Unlike sliding tile puzzles (like the 15-puzzle) where mistakes can often be undone, Sokoban features "dead states"—configurations from which the goal becomes unreachable. Research indicates that in random configurations, up to 73.8% of reachable states can be dead. These deadlocks fall into two categories:

- **Static Deadlocks:** Pushing a box into a corner or against a wall where it can never be moved again.
- **Dynamic Deadlocks:** Complex configurations, such as a 2×2 cluster of boxes (frozen blocks), which render the involved boxes immovable regardless of the agent's position.

This high density of dead ends makes unguided search algorithms like Breadth-First Search (BFS) infeasible, as they waste vast computational resources exploring subtrees that have already become unsolvable.

Failure of Simple Heuristics Standard search heuristics often fail in Sokoban due to the need for "counter-intuitive moves". A simple Hill-Climbing heuristic, which tries to minimize the total distance of boxes to goals, is rarely sufficient. Solving a level often requires pushing a box temporarily *away* from its goal to clear a path for another box or the agent. These deviations from the local gradient create local optima that trap standard greedy algorithms.

Branching Factor and Depth While the immediate branching factor (number of legal moves) is relatively low, the effective search depth is immense. Solutions often require hundreds of steps. Furthermore, the state space is not just a function of grid size but of the combinatorial arrangement of boxes. The complexity is compounded by the fact that the agent's movement is constrained by the very boxes it must move, creating a tightly coupled dependency between the agent's reachability and the box configuration.

2.2 Implementation Details (Task 1)

To model this problem for the required AI techniques (A^* and PDDL planning), I implemented a robust environment in `Sokoban.py`. The implementation focuses on reducing the effective state space and detecting deadlocks early to make the search computationally viable.

2.2.1 Canonical State Representation

A naive representation of the state, tracking the raw (x, y) coordinates of the player and every box, results in a massive explosion of duplicate states. For example, if the player moves one step to the right in an empty room without pushing a box, the logical state of the puzzle (in terms of box configuration) has not changed, yet a naive solver would treat it as a new node.

To address this, I implemented a canonical state representation. Instead of storing the player's exact position, I compute the set of all squares the player can reach via BFS without pushing a box. The state is stored as a `namedtuple` containing:

- `boxes`: A `frozenset` of box coordinates. Using a set provides $O(1)$ lookups and order independence, ensuring that the state is identical regardless of the order in which boxes are tracked.
- `player`: A canonical "anchor" position, defined as the top-left-most coordinate (minimum x, then minimum y) within the reachable area.

This abstraction collapses all equivalent player positions into a single state, significantly reducing the branching factor and graph size.

2.2.2 Static Deadlock Pre-computation

To prune the search space efficiently, the `SokobanLevel` class performs a static analysis upon initialization using a Reverse BFS. The algorithm starts with boxes placed at all goal locations and attempts to "pull" them backwards into valid empty spaces. Any floor tile that is never reached during this reverse exploration is marked as a "Dead Square".

During the forward search in A^*, any action that results in a box landing on a Dead Square is immediately pruned. This prevents the solver from even considering moves that push boxes into corners or unreachable corridors.

2.2.3 Dynamic Deadlock Detection

While static analysis handles wall-based traps, it cannot detect deadlocks formed by box-to-box interactions. I implemented a runtime check in the `get_successors` function to identify 2×2 "frozen" blocks. Whenever a box is pushed, the system checks if it has formed a 2×2 square of obstacles (walls or other boxes). If such a block is formed and any involved box is not on a goal, the state is discarded as a dynamic deadlock.

2.2.4 Interface Design

The implementation exposes a modular interface designed to support both the custom A^* solver and the PDDL generator:

- `get_successors(state)`: Returns a list of valid `(action, new_state)` tuples, applying all pruning logic described above.
- `is_goal(state)`: A simple set containment check to verify if all box positions are currently within the set of goal positions.

3 Task 2.1: Implementation of A*

The core of the solver is a custom implementation of the A^* search algorithm, located in `Astar.py`. This implementation is designed to be modular, allowing for easy swapping of heuristics and problem definitions.

3.1 Algorithm Design

The implementation follows the standard best-first search paradigm with several key features to ensure efficiency and correctness in the Sokoban domain:

- **Priority Queue:** We use Python's `heapq` module to manage the open set. The priority queue stores tuples in the format `(f_score, push_order, state, path)`. The `push_order` acts as a tie-breaker, ensuring that among nodes with equal f-scores, the algorithm prefers the one generated earlier (FIFO behavior for ties), which adds stability to the search.

- **Duplicate Elimination:** A `visited` set stores every expanded state. Before pushing a neighbor to the priority queue, the algorithm checks if it has already been visited. This is crucial for Sokoban, as many different sequences of moves can lead to the identical board configuration.
- **Weighted A*:** The cost function is defined as $f(n) = g(n) + w \times h(n)$. The implementation accepts a `weight` parameter. While standard A^* ($w = 1$) guarantees optimality, using $w > 1$ (Weighted A^*) essentially turns the algorithm into "Best-first Search with a bias towards the goal". In PSPACE-complete problems like Sokoban, sacrificing strict optimality for speed is often necessary to find solutions within a reasonable time frame.
- **Resource Limits:** To prevent the process from hanging on unsolvable or extremely difficult instances, the algorithm includes a strict `limit` on the number of expanded nodes.

4 Task 2.2: Heuristics Description

The effectiveness of A^* is entirely dependent on the quality of the heuristic function $h(n)$. I implemented several heuristics ranging from simple distance metrics to complex bipartite matching algorithms. All heuristics are located in the `heristics/` directory.

4.1 Basic Distance Metrics

4.1.1 Manhattan Distance (`Manhattan_Distance.py`)

This is the simplest heuristic. It calculates the sum of the Manhattan distances ($|x_1 - x_2| + |y_1 - y_2|$) from each box to its *nearest* goal.

- **Pros:** Extremely fast to compute ($O(1)$ per box).
- **Cons:** It ignores walls and obstacles, leading to severe underestimation of the true cost. It often guides the search into dead ends (e.g., trying to push a box through a wall).

4.1.2 Real Maze Distance (`Real_Maze_Distance.py` & `Fast_Distance.py`)

To improve upon Manhattan distance, these heuristics replace the geometric distance with the true "Maze Distance"—the length of the shortest path on the grid considering walls.

- **Implementation:** Upon initialization, the algorithm performs a Breadth-First Search (BFS) starting simultaneously from all goal squares to flood-fill the grid. This creates a cached map where every tile knows its true distance to the nearest goal.
- **Accuracy:** This is significantly more informative than Manhattan distance because it accounts for the layout of the maze. However, it still treats each box independently, ignoring the fact that boxes might block each other.

4.2 Matching-Based Heuristics

The basic metrics above fail to account for the "assignment problem": if two boxes are close to the *same* goal, basic heuristics assumes both can go there, even though only one fits. Matching heuristics solve this by assigning specific boxes to specific goals.

4.2.1 Greedy Matching (`Exact_Distance.py`)

This heuristic computes the Maze Distance from every box to every goal and then performs a greedy assignment.

- **Logic:** It creates a list of all possible box-to-goal pairings, sorts them by distance, and iteratively assigns the closest available pair until all boxes are matched.
- **Trade-off:** This is faster than finding the optimal matching but may produce suboptimal total costs. It provides a good middle ground between speed and accuracy.

4.2.2 Optimal Assignment / Hungarian Algorithm (`Hungarian_BFS.py`)

This represents the most mathematically rigorous heuristic implemented.

- **Logic:** It constructs a full cost matrix C where C_{ij} is the BFS distance from box i to goal j. It then solves the **Linear Sum Assignment Problem** (LSAP) to find the assignment that minimizes the total displacement.
- **Implementation:** I utilized `scipy.optimize.linear_sum_assignment` (which implements the Hungarian Algorithm) to solve this matrix.
- **Pros:** It provides the most accurate lower bound on the remaining effort (excluding box-box collisions).
- **Cons:** The computation of the full cost matrix and the $O(n^3)$ Hungarian algorithm makes this heuristic computationally expensive per node.

4.2.3 Recursive Optimal Assignment (`New_Real_Maze_Distance.py`)

This is a custom implementation of the optimal assignment logic without external dependencies like SciPy. It uses a recursive backtracking algorithm with memoization to find the minimum cost assignment. While functionally similar to the Hungarian method, the exponential nature of recursion makes it viable only for puzzles with a small number of boxes.

4.2.4 Satisficing with Penalty (`Satificing.py`)

This heuristic is designed specifically for Weighted A^* to encourage "safe" play.

- **Logic:** It combines the Greedy Matching strategy (from `Exact_Distance`) with a penalty term.
- **Penalty:** It scans the board for adjacent boxes. If two boxes are next to each other and not on goals, it adds a penalty (cost increase) to the heuristic value.
- **Purpose:** This discourages the agent from pushing boxes into clusters, which are high-risk states for dynamic deadlocks (frozen blocks). This makes the search more "cautious" and helps avoid complex deadlock states.

5 Task 2.2: Planning with PDDL

For the second AI technique, I modeled the Sokoban problem using the Planning Domain Definition Language (PDDL). This allows us to leverage powerful, domain-independent planners like Fast Downward and LAMA to find solutions without writing custom search algorithms.

5.1 Domain Definition

The domain logic is defined in `domain.pddl`. I used a standard `STRIPS` formulation with typing.

5.1.1 Types and Predicates

The world is defined by two primary types: `location` (representing grid cells) and `direction`. The state of the world is described using the following predicates:

- `(at-player ?l)`: The player is currently at location `?l`.
- `(at-box ?l)`: A box is currently at location `?l`.
- `(clear ?l)`: Location `?l` is free of dynamic obstacles (player or boxes). This predicate simplifies precondition checks.
- `(adjacent ?from ?to ?d)`: A static predicate defining the grid topology. It states that location `?to` is immediately reachable from `?from` in direction `?d`.

5.1.2 Actions

The domain defines two atomic actions:

1. `(:action move ...)`: The player moves from one clear adjacent square to another.
2. `(:action push ...)`: The player pushes a box. This action requires three aligned locations: the player's current spot, the box's current spot, and the destination spot. The effect updates the positions of both the player and the box, and toggles the `clear` status of the involved cells.

5.2 Problem Generation (`PDDL_Generator.py`)

Writing PDDL problem files manually for each level is error-prone and tedious. To automate this, I implemented a `PDDLGenerator` class that translates the Python `SokobanLevel` object into a valid PDDL problem file.

5.2.1 Grid Translation

The generator iterates through the grid dimensions ($W \times H$). For every coordinate (x, y) that is not a wall:

1. It creates a PDDL object `pos_x_y` of type `location`.
2. It computes valid neighbors. If cell $(x + 1, y)$ is also not a wall, it generates the fact `(adjacent pos_x_y pos_x+1_y right)`. This effectively "bakes" the map topology and wall constraints directly into the static facts of the problem file.

5.2.2 State Initialization

The generator extracts the initial coordinates of the player and boxes from the level object to populate the `(:init ...)` block. It explicitly adds `(clear ...)` facts for every valid floor tile that does not initially contain a box. Finally, it iterates through the level's goal set to construct the `(:goal (and ...))` condition, requiring a box to be present at every target coordinate.

5.3 Planner Integration

The solution is integrated into the main pipeline via `main.py`. Instead of implementing a planner from scratch, the system calls external binaries for state-of-the-art planners:

- **Fast Downward:** Configured with a greedy best-first search using the FF heuristic (`lazy_greedy([ff()], preferred=[ff()])`). This configuration prioritizes speed over optimality.
- **LAMA (First):** A highly efficient satisficing planner.

The script automatically invokes these planners on the generated `.pddl` files, captures their standard output to parse metrics (expanded nodes, time), and reads the generated `sas_plan` file to verify the solution length.

6 Task 3: Experimental Results

In this section, we evaluate the performance of our A^* implementation and the PDDL planners. The experiments were conducted on two distinct datasets to measure scalability and robustness:

1. **Easy Benchmark:** A custom set of 22 simplified AI generated levels designed to verify correctness and analyze baseline overhead.
2. **Standard Benchmark:** The first 5 levels of the original Sokoban test suite, which represent "real-world" difficulty with significantly higher branching factors and solution depths.

6.1 Performance on Easy Benchmark

The experiments on the simplified dataset (22 levels) reveal the baseline overhead of the different approaches and demonstrate the impact of heuristic quality even on trivial problems. Table 1 summarizes the key performance metrics.

Algorithm	Success (%)	Time (s)	Nodes	Sol. Len.
A^* (Exact Distance)	86.36	0.00	8.82	5.26
A^* (Fast Distance)	86.36	0.01	36.05	5.26
A^* (Hungarian)	86.36	0.01	32.36	5.26
A^* (Manhattan)	86.36	0.01	36.05	5.26
A^* (New Real Maze)	86.36	0.01	9.23	5.26
A^* (Real Maze)	86.36	0.01	36.05	5.26
A^* (Satisficing)	86.36	**0.00**	**6.50**	**5.16**
Fast Downward	86.36	2.83	2172.05	17.95
LAMA First	**90.91**	0.62	7093.82	25.45

Table 1: Average metrics on the Easy Benchmark (22 levels). The specialized A^* implementation is orders of magnitude faster than general planners due to minimal overhead.

6.1.1 Analysis of Failed Instances

While the overall success rates are high, the distribution of failures highlights distinct trade-offs between the specialized A^* solvers and the general-purpose PDDL planners. Table 2 isolates the specific levels that resulted in failure for at least one algorithm.

Level	Name	A^* Status	Fast Downward	LAMA First
Level 7	Level 7	**Success**	Failed (Timeout)	**Success**
Level 12	The Cross	**Success**	Failed	Failed
Level 13	Around the Bend	Failed	Failed	Failed
Level 16	Double Trouble	Failed	**Success**	**Success**
Level 21	The Wall	Failed	**Success**	**Success**

Table 2: Breakdown of failed levels. A^* struggles with specific deadlock configurations (Levels 16, 21), while general planners struggle with search depth on highly branched puzzles (Levels 7, 12).

Planner Scalability vs. Specialized A* General planners struggled with specific branching structures. Fast Downward timed out on Level 7 and failed Level 12, whereas specialized A^* solved Level 7 in just 0.04s (98 nodes). Even when successful (e.g., LAMA on Level 7), the general planners required orders of magnitude more expansions (119k) compared to the specialized approach.

Deadlock Detection Trade-offs Levels 16 and 21 exposed a critical limitation in the specialized heuristics: all A^* variants failed, identifying these solvable states as deadlocks (infinite cost) at the root. In contrast, the PDDL planners successfully navigated these levels. Level 13, however, caused a universal failure, with its corridor structure blocking both distance-based heuristics and landmark search.

6.1.2 Heuristic Pruning Efficiency

The behavior on failed levels (16 and 21) illustrates the aggressive pruning of complex heuristics:

- **Wandering:** Simple heuristics (`Manhattan`) exhaustively searched the space (expanding 56-455 nodes) before exhausting memory or time.
- **Instant Pruning:** Complex heuristics (`Exact Distance`, `Satisficing`) terminated after expanding only **1 node**.

While this pruning caused false negatives on these specific levels, it demonstrates the complex heuristics' ability to instantly reject deadlock states, a crucial trait for efficiency in larger search spaces.

6.2 Performance on Real Benchmark

The transition to original Sokoban puzzles (Levels 1-5) reveals the scalability limits of the tested algorithms. The experiments were conducted with a 200,000 node limit for the initial levels (1-3) and a 1,000,000 node limit for the harder instances (4-5). Table 3 highlights the performance on Level 1, the only level in this set that was successfully solved by the algorithms.

6.2.1 Algorithmic Efficiency on Solvable Levels

Level 1 serves as a critical benchmark for search efficiency on non-trivial problems.

- **Orders of Magnitude Speedup:** The specialized A^* implementation with the `Exact Distance` heuristic solved the level in **0.05 seconds** expanding only 553 nodes. In stark contrast, the domain-independent planner **Fast Downward** required nearly **8 minutes** (474s) and expanded over **35 million nodes** to find a solution.

Algorithm	Status	Time (s)	Nodes	Sol. Len.
A^* (Exact Distance)	**Success**	**0.05**	**553**	117
A^* (Fast Distance)	**Success**	0.07	798	117
A^* (Hungarian)	**Success**	**0.05**	**553**	117
A^* (Manhattan)	**Success**	3.50	35,381	103
A^* (New Real Maze)	**Success**	0.16	**553**	117
A^* (Real Maze)	**Success**	0.08	798	117
A^* (Satisficing)	**Success**	4.05	35,038	**97**
Fast Downward	**Success**	474.72	35,074,333	422
LAMA First	Timeout	500.00	-	-

Table 3: Performance on original Level 1. The specialized A^* solvers are orders of magnitude more efficient than general planners. Note that simpler heuristics (Satisficing, Manhattan) found shorter paths despite expanding more nodes.

- **Solution Quality Anomaly:** Interestingly, the simple heuristics and the "Satisficing" variant found shorter paths (97-103 steps) than the complex, more informed heuristics (117 steps). This suggests that the `Exact Distance` and `Hungarian` heuristics, while extremely efficient at pruning the search space, may guide the agent toward a specific, suboptimal solution corridor.
- **Planner Inefficiency:** Fast Downward produced a significantly longer plan (422 steps), demonstrating the "satisficing" nature of its greedy search, which prioritizes finding *any* solution over an optimal one.

6.2.2 The Complexity Cliff (Levels 2-5)

Beyond Level 1, the problem complexity exceeds the capabilities of all tested configurations, resulting in a universal failure rate.

- **Resource Exhaustion:** For Levels 2 and 3, all algorithms hit the **200,000 node limit** without finding a solution. Even when the limit was raised to **1,000,000 nodes** for Levels 4 and 5, the A^* solvers failed to converge.
- **Planner Timeout:** LAMA First and Fast Downward consistently timed out (500s limit) or failed on these levels, confirming that the difficulty stems from the puzzle's combinatorial structure rather than just heuristic implementation.

These results indicate that while specialized heuristics provide a massive advantage on solvable instances, raw search power alone is insufficient for difficult Sokoban levels. Solving these instances likely requires advanced techniques to further prune the search space.

7 Conclusion

This study evaluated the performance of specialized A^* solvers against domain-independent PDDL planners across a spectrum of Sokoban puzzle complexities. The results demonstrate a clear trade-off between initialization overhead, heuristic quality, and search scalability.

7.1 Specialized Solvers vs. General Planners

The specialized Python-based A^* implementation consistently outperformed general planners on solvable instances. On the "Real" benchmark (Level 1), the A^* algorithm using the `Exact`

`Distance` heuristic found a solution in **0.05 seconds**, whereas the Fast Downward planner required **474 seconds** to solve the same instance. This performance gap is driven by two factors:

- **Initialization Overhead:** General planners incur significant startup costs (0.1s - 0.6s) to ground the domain, which dominates the runtime for trivial problems.
- **Search Efficiency:** Specialized heuristics like `Hungarian` and `Exact Distance` provide aggressive pruning, reducing the search space by orders of magnitude compared to the greedy search of Fast Downward.

7.2 The Impact of Heuristic Quality

Our analysis reveals that "smarter" heuristics are not always strictly better. While the computationally expensive `Exact Distance` heuristic minimized node expansion, it occasionally produced longer, suboptimal paths (117 steps) compared to simpler heuristics like `Manhattan` (103 steps) or `Satisficing` (97 steps) on complex maps. This suggests that aggressive pruning can inadvertently discard shorter solution paths that require traversing "high-cost" intermediate states.

Furthermore, the failure modes on the Easy benchmark highlight the value of sophisticated pruning. Complex heuristics instantly identified impossible configurations (deadlocks) in Levels 16 and 21, expanding only a single node, while simpler heuristics wasted computational resources "wandering" the map.

7.3 Scalability Limits

Despite the success on Level 1, all algorithms hit a "complexity cliff" on Real Levels 2 through 5. Neither the specialized heuristics nor the PDDL planners could solve these instances within the 1,000,000 node or 500-second limits. This indicates that for realistic, highly combinatorial Sokoban levels, standard state-space search—even with strong heuristics—is currently insufficient.

However, given that Sokoban is known to be PSPACE-complete, these failures may simply reflect resource constraints rather than fundamental algorithmic flaws. Future work could explore whether significantly extending runtime and memory limits on high-performance hardware would allow these standard approaches to converge or if advanced domain-specific techniques (e.g., deadlock pattern databases, tunnel macros) are strictly necessary to solve these harder instances effectively.

8 How to run

This section details the necessary steps to set up the execution environment, install dependencies, and run the experiments to reproduce the results presented in this report.

8.1 Environment Setup and Dependencies

The project is developed using **Python 3**. To ensure all necessary Python libraries are installed, a `requirements.txt` file is provided in the root directory of the repository.

To set up the environment, navigate to the project root and execute the following command:

```
pip install -r requirements.txt
```

8.2 External Planners Configuration

As part of Task 2.2, this project utilizes PDDL modeling solved by external planners. The setup for these is handled as follows:

- **Fast Downward:** The planner binaries are included directly in the repository under the `fast-downward/` directory. No additional installation is required for this planner.
- **LAMA (LAMA-First):** This planner is managed via a wrapper script named `lama-first` located in the root directory. Upon the first execution, the script utilizes `planutils` to automatically check for and install the LAMA planner if it is not already present on the system.

8.3 Execution and Reproducing Results

The project includes two entry points to run the experiments, depending on the desired difficulty of the problem instances:

1. **Standard Experiments:** To run the full pipeline on the original Sokoban benchmarks (defined in `sokoban_levels.txt`), execute:

   ```
   python main.py
   ```

2. **Simplified Experiments:** To run the pipeline on a set of easier, AI-generated levels (defined in `ez_levels.txt`), execute:

   ```
   python main_ez.py
   ```

8.4 Output and Metrics

Real-time progress is displayed in the console. Upon completion, the detailed experimental data (execution time, node expansion, memory usage) is exported to CSV files (`experiment_data.csv` or `experiment_data_ez.csv`) for analysis. Detailed execution logs are saved to `experiment_log.txt` or `experiment_log_ez.txt`.

9 GenAI Acknowledgment

Generative AI was used as an aid in the project and even for writing this report